

NAGALAND UNIVERSITY

(A Central University Established by an Act of Parliament 1989)

SCHOOL OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

Kohima Campus, Nagaland – 797004



Assignment 1

ITB301: Object Oriented Programming

Submitted By:
Soushnabh Bagchi
Roll / ID: 2505264
Semester: 3rd Sem

Submitted To:
Er Sudipto Patowary
Assistant Professor
Dept. of IT

Date: September 18, 2026

1. Write a program to convert decimal to binary number using function.

Code:

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  class ToBinary {
7  public:
8      int n;
9      string ans = "";
10
11     void binary_direct() {
12         for (int i = 31; i >= 0; i--) {
13             ans += to_string((n >> i) & 1);
14         }
15     }
16
17     void binary_factor() {
18         int temp = n;
19         while (temp > 0) {
20             ans = to_string(temp % 2) + ans;
21             temp /= 2;
22         }
23     }
24 };
25
26 int main() {
27     ToBinary b;
28     cout << "Enter n: ";
29     cin >> b.n;
30
31     cout << "The binary representation is: " << endl;
32
33     cout << "1. Using bitshifting." << endl;
34     b.binary_direct();
35     cout << b.ans << endl;
36     b.ans = "";
37
38     cout << "2. Using factorization." << endl;
```

C++

```
39     b.binary_factor();
40     cout << b.ans << endl;
41
42     return 0;
43 }
```

Output:

[illegible]

```
1 Enter n: 123456789
2 The binary representation is:
3 1. Using bitshifting.
4 00000111010110111100110100010101
5 2. Using factorization.
6 111010110111100110100010101
```

2. Write a program to using switch statement for the following:

- Root of a given number using Newton-Raphson Method.
- Secant Method
- Fixed Point Iteration

Code:

```
1  #include <cmath>
2  #include <iomanip>
3  #include <iostream>
4
5  using namespace std;
6
7  class ComputeRoots {
8      double fx(double x) { return pow(x, root) - n; }
9      double gx(double x) {
10         return ((root - 1) * x + (n / pow(x, root - 1))) / root;
11     }
12     double ddx(double x) { return root * pow(x, root - 1); }
13     int approx_root() {
14         long long l = 0, r = n, m;
```

```

15         while (l <= r) {
16             m = (l + r) / 2;
17             if (pow(m, root) <= n) {
18                 l = m + 1;
19             } else {
20                 r = m - 1;
21             }
22         }
23         return m;
24     }
25
26 public:
27     double n;
28     int iters;
29     int root;
30     double sol;
31
32     void set_sol() { sol = approx_root(); }
33
34     void newton_raphson(int iters) {
35         if (sol == 0) {
36             return;
37         }
38         sol = sol - (fx(sol) / ddx(sol));
39         if (iters == 1) {
40             return;
41         }
42         newton_raphson(iters - 1);
43     }
44     void secant(int iters) {
45         if (sol == 0) {
46             return;
47         }
48         int b = sol + 1;
49         sol = ((b * fx(sol)) - (sol * fx(b))) / (fx(sol) - fx(b));
50         if (iters <= 2) {
51             return;
52         }
53         secant(iters - 1);
54     }
55     void fixedpointiter(int iters) {
56         if (sol == 0) {

```

```

57         return;
58     }
59     sol = gx(sol);
60     if (iters == 1) {
61         return;
62     }
63     fixedpointiter(iters - 1);
64 }
65 };
66
67 int main() {
68     ComputeRoots c;
69     cout << "Enter n: ";
70     cin >> c.n;
71     cout << "Enter the root: ";
72     cin >> c.root;
73     cout << "Enter the number of iterations: ";
74     cin >> c.iters;
75     c.set_sol();
76
77     char method;
78     cout << "Enter the method to be used:" << endl;
79     cout << "(N)ewton, (S)ecant or (F)ixed Point: ";
80     cin >> method;
81     switch (method) {
82         case 'N': {
83             c.newton_raphson(c.iters);
84             break;
85         }
86         case 'S': {
87             c.secant(c.iters);
88             break;
89         }
90         case 'F': {
91             c.fixedpointiter(c.iters);
92             break;
93         }
94         default: {
95             cout << "Invalid Input " << "\"" << method << "\"" <<
96             endl;
97             return 1;
98         }
99     }

```

```
98     }  
99  
100    cout << "The root is: " << fixed << setprecision(15) << c.sol <<  
    endl;  
101    return 0;  
102 }
```

Output:

```
1 Enter n: 64  
2 Enter the root: 2  
3 Enter the number of iterations: 5  
4 Enter the method to be used:  
5 (N)ewton, (S)ecant or (F)ixed Point: N  
6 The root is: 8.000000000000000
```

```
1 Enter n: 99  
2 Enter the root: 3  
3 Enter the number of iterations: 3  
4 Enter the method to be used:  
5 (N)ewton, (S)ecant or (F)ixed Point: N  
6 The root is: 4.626066084823458
```

```
1 Enter n: 759237165  
2 Enter the root: 5  
3 Enter the number of iterations: 10  
4 Enter the method to be used:  
5 (N)ewton, (S)ecant or (F)ixed Point: S  
6 The root is: 59.713907599367566
```

```
1 Enter n: 1122334455  
2 Enter the root: 4  
3 Enter the number of iterations: 7  
4 Enter the method to be used:  
5 (N)ewton, (S)ecant or (F)ixed Point: F  
6 The root is: 183.033495564811233
```